LAPORAN PRAKTIKUM
PEMROGRAMAN MOBILE
MODUL 3

BUILD A SCROLLABLE LIST

Oleh:

Arinda Dita Pratiwi NIM. 2410817120018

PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
APRIL 2026

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM PEMROGRAMAN MOBILE
MODUL 3

Laporan Praktikum Pemrograman Mobile Modul 3: Build a Scrollable List ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Arinda Dita Pratiwi
NIM : 2410817120018

Menyetujui, Asisten Praktikum	Mengetahui, Dosen Penanggung Jawab Praktikum
Muhammad Erza Raihan NIM. 2310817210027	Irham Maulani Abdul Gani, S.Kom., M.Kom. NIP. 199710312025061009

DAFTAR ISI

LEMBAR PENGESAHAN 2
DAFTAR ISI 3
DAFTAR GAMBAR 4
DAFTAR TABEL 5
SOAL 1 6
A. Source Code 9
B. Output Program 59
C. Pembahasan 63
SOAL 2 63
A. Pembahasan 73
ADDITIONAL QUESTION ..... **Error! Bookmark not defined.**
Daftar Pustaka ..... **Error! Bookmark not defined.**
TAUTAN GIT 77

DAFTAR GAMBAR

Gambar 1. Tampilan Awal Aplikasi 7
Gambar 2. Tampilan Pilihan Persentase Tip.....**Error! Bookmark not defined.**
Gambar 3. Tampilan Aplikasi Setelah DIjalankan .....**Error! Bookmark not defined.**
Gambar 4. Screenshot Hasil Jawaban Soal 1 XML.....**Error! Bookmark not defined.**
Gambar 5. Screenshot Hasil Jawaban Soal 1 XML.....**Error! Bookmark not defined.**
Gambar 6. Screenshot Hasil Jawaban Soal 1 XML.....**Error! Bookmark not defined.**
Gambar 7. Screenshot Hasil Jawaban Soal 1 Compose.....**Error! Bookmark not defined.**
Gambar 8. Screenshot Hasil Jawaban Soal 1 Compose.....**Error! Bookmark not defined.**
Gambar 9. Screenshot Hasil Jawaban Soal 1 Compose.....**Error! Bookmark not defined.**

DAFTAR TABEL

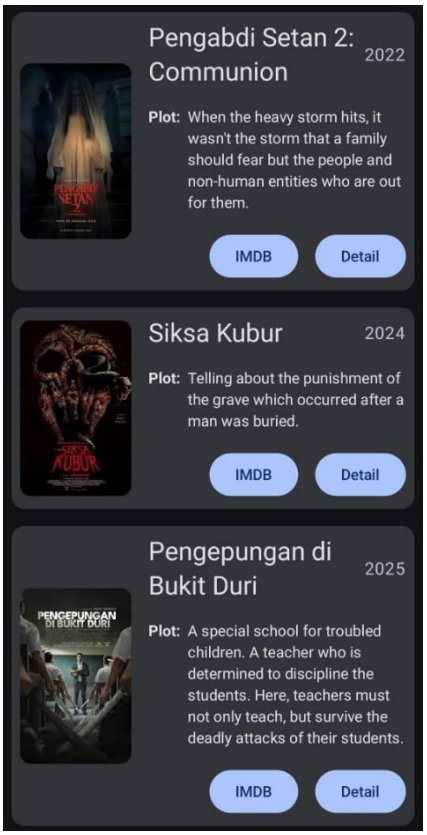
Tabel 1. Source Code MainActivity.kt XML .....**Error! Bookmark not defined.**
Tabel 2. Source Code activity_main.xml.....**Error! Bookmark not defined.**
Tabel 3. Source Code string.xml.....**Error! Bookmark not defined.**
Tabel 4. Source Code MainActivity.kt Compose .....**Error! Bookmark not defined.**

SOAL 1

Buatlah sebuah aplikasi Android menggunakan XML dan Jetpack Compose yang dapat menampilkan list dengan ketentuan berikut:

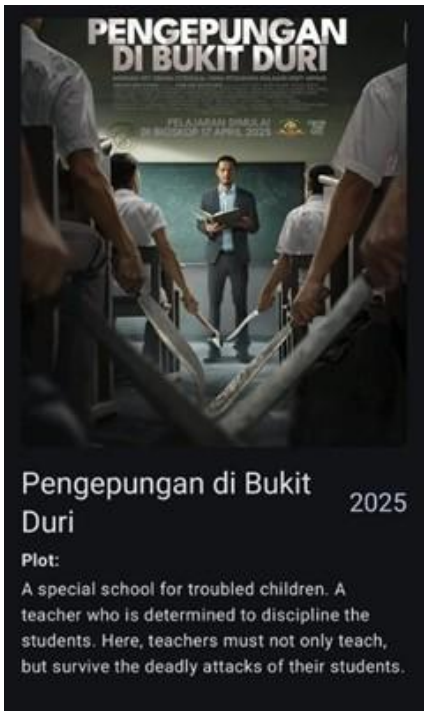
1. List menggunakan fungsi RecyclerView (XML) dan LazyColumn (Compose)
2. List paling sedikit menampilkan 5 item. Tema item yang ingin ditampilkan bebas
3. Item pada list menampilkan teks dan gambar sesuai dengan contoh di bawah
4. Terdapat 2 button dalam list, dengan fungsi berikut:
   a. Button pertama menggunakan intent eksplisit untuk membuka aplikasi atau browser lain
   b. Button kedua menggunakan Navigation component untuk membuka laman detail item
5. Sudut item pada list dan gambar di dalam list melengkung atau rounded corner menggunakan Radius
6. Saat orientasi perangkat berubah/dirotasi, baik ke potrait maupun landscape, aplikasi responsif dan dapat menunjukkan list dengan baik. Data di dalam list tidak boleh hilang
7. Aplikasi menggunakan arsitektur *single activity* (satu activity memiliki beberapa fragment)
8. Aplikasi berbasis XML harus menggunakan ViewBinding
9. Aplikasi mendukung Multi-Language (bahasa indonesia dan inggris)
10. Terdapat 1 halaman/screen untuk melakukan pergantian bahasa
11. Gunakan Icol dari librari ‘material’ (material1, material2, material3) untuk membuat tombol yang mengarahkan ke halaman untuk melakukan pergantian bahasa, dan letakkan di bagian header/heading homescreen utama
12. Di atas Scrollable list, terdapat sebuah “Item Highlight” yang menggunakan Carousel atau LazyRow dengan ketentuan:
    - Item bisa bergeser secara finite atau infinite loop sebanyak jumlah item yang ada pada scrollable-list

UI item list harus berisi 1 gambar, 2 button (intent eksplisit dan navigasi), dan 2 baris teks dam setiap baris memiliki 2 teks yang berbeda. Diusahakan agar desain UI item list menyerupai UI berikut:

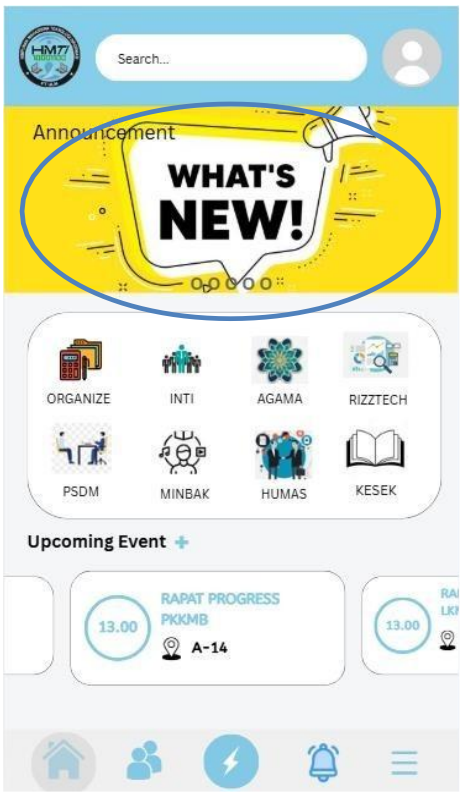


Gambar 1. Contoh UI List

Desain UI laman detail bebas, tetapi diusahakan untuk mengikuti kaidah desain Material Design dan data item ditampilkan penuh di laman detail seperti contoh berikut:



Gambar 2. Contoh UI Detail



Gambar 3. Contoh Penerapan LazyRow/Carousel

Item yang bergeser bisa secara finite (mentok kiri kanan) atau secara infinite loop (misal dari item terakhir '5' kembali ke item awal '1')

A. Source Code

XML

MainActivity.kt

Tabel 1. Source Code MainActivity.kt

```
package com.example.songpediaxml

import android.content.Context
import android.content.res.Configuration
import android.os.Bundle
import androidx.appcompat.app.AppCompatActivity
import androidx.appcompat.app.AppCompatDelegate
import com.example.songpediaxml.databinding.ActivityMainBinding
import java.util.Locale


class MainActivity : AppCompatActivity() {



    private lateinit var binding: ActivityMainBinding



    override fun onCreate(savedInstanceState: Bundle?) {

        super.onCreate(savedInstanceState)


AppCompatDelegate.setDefaultNightMode(AppCompatDelegate.MODE_NIGHT_NO)


        val prefs = getSharedPreferences("settings", Context.MODE_PRIVATE)
        if (!prefs.contains("language")) {

            prefs.edit().putString("language", "id").apply()

        }

        val lang = prefs.getString("language", "id") ?: "id"

```

```
24
25          val locale = Locale(lang)
26
27          Locale.setDefault(locale)
28
29          val config = Configuration(resources.configuration)
30
31          config.setLocale(locale)
32
33          resources.updateConfiguration(config,
34  resources.displayMetrics)
35
36
37          binding                                                =
38  ActivityMainBinding.inflate(layoutInflater)
39          setContentView(binding.root)
40
41      }
42
43  }
44
```

HomeFragment.kt

Tabel 2. Source Code HomeFragment.kt

```
1   package com.example.songpediaxml
2
3   import android.content.Intent
4   import android.net.Uri
5   import android.os.Bundle
6   import android.view.LayoutInflater
7   import android.view.View
8   import android.view.ViewGroup
9   import androidx.fragment.app.Fragment
10  import androidx.fragment.app.activityViewModels
11  import androidx.navigation.fragment.findNavController
12  import androidx.recyclerview.widget.LinearLayoutManager
13  import
    com.example.songpediaxml.databinding.FragmentHomeBinding
14
15  class HomeFragment : Fragment() {
16
17      private var _binding: FragmentHomeBinding? = null
18      private val binding get() = _binding!!
```

```
    private val viewModel: SongViewModel by activityViewModels()
    private lateinit var songAdapter: SongAdapter
    private lateinit var carouselAdapter: CarouselAdapter

    override fun onCreateView(
        inflater: LayoutInflater,
        container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View {
        _binding = FragmentHomeBinding.inflate(inflater, container, false)
        return binding.root
    }

    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
        super.onViewCreated(view, savedInstanceState)

        songAdapter = SongAdapter(
            songs = emptyList(),
            onDetailClick = { song ->
                viewModel.selectSong(song)
findNavController().navigate(R.id.action_home_to_detail)
            },
            onSpotifyClick = { song ->
                val intent = Intent(Intent.ACTION_VIEW, Uri.parse(song.spotifyUrl))
                startActivity(intent)
            }
        )

        carouselAdapter = CarouselAdapter(emptyList())

        binding.rvSongs.layoutManager = LinearLayoutManager(requireContext())
        binding.rvSongs.adapter = songAdapter

        binding.rvCarousel.layoutManager = LinearLayoutManager(
            requireContext(),
            LinearLayoutManager.HORIZONTAL,
            false
        )
        binding.rvCarousel.adapter = carouselAdapter

```

```
        viewModel.songs.observe(viewLifecycleOwner)      { songs ->
            songAdapter = SongAdapter(
                songs = songs,
                onDetailClick = { song ->
                    viewModel.selectSong(song)
findNavController().navigate(R.id.action_home_to_detail)
                },
                onSpotifyClick = { song ->
                    val intent = Intent(Intent.ACTION_VIEW, Uri.parse(song.spotifyUrl))
                    startActivity(intent)
                }
            )
            binding.rvSongs.adapter = songAdapter
            carouselAdapter = CarouselAdapter(songs)
            binding.rvCarousel.adapter = carouselAdapter
        }

        binding.btnLanguage.setOnClickListener {
findNavController().navigate(R.id.action_home_to_language)
        }
    }

    override fun onDestroyView() {
        super.onDestroyView()
        _binding = null
    }
}
```

DetailFragment.kt

Tabel 3. Source Code DetailFargment.kt

```
package com.example.songpediaxml

import android.content.Context
import android.os.Bundle
import android.view.LayoutInflater
import android.view.ViewGroup
import android.view.View
import androidx.fragment.app.Fragment
import androidx.fragment.app.activityViewModels
import com.bumptech.glide.Glide
```

```
import com.bumptech.glide.load.resource.bitmap.RoundedCorners
import com.example.songpediaxml.databinding.FragmentDetailBinding

class DetailFragment : Fragment() {

    private var _binding: FragmentDetailBinding? = null
    private val binding get() = _binding!!
    private val viewModel: SongViewModel by activityViewModels()

    override fun onCreateView(
        inflater: LayoutInflater,
        container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View {
        _binding = FragmentDetailBinding.inflate(inflater, container, false)
        return binding.root
    }

    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
        super.onViewCreated(view, savedInstanceState)

        val prefs = requireContext().getSharedPreferences("settings", Context.MODE_PRIVATE)
        val lang = prefs.getString("language", "id")

        viewModel.selectedSong.observe(viewLifecycleOwner) { song ->
            binding.tvDetailTitle.text = song.title
            binding.tvDetailArtist.text = song.artist
            binding.tvDetailYear.text = song.year.toString()
            binding.tvDetailAlbum.text = song.album
            binding.tvDetailGenre.text = song.genre

            if (lang == "id") {
                binding.tvDescriptionLabel.text = getString(R.string.label_description)
                binding.tvDetailDescription.text = song.descriptionId
            } else {
```

```
                binding.tvDescriptionLabel.text = getString(R.string.label_description)
                binding.tvDetailDescription.text = song.descriptionEn
            }

            Glide.with(requireContext())
                .load(song.imageRes)
                .transform(RoundedCorners(32))
                .into(binding.imgDetailAlbum)
        }
    }

    override fun onDestroyView() {
        super.onDestroyView()
        _binding = null
    }
}
```

LanguageFragment.kt

Tabel 4. Source Code LanguageFragment.kt

```
package com.example.songpediaxml

import android.content.Context
import android.content.res.Configuration
import android.os.Bundle
import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import androidx.fragment.app.Fragment
import com.example.songpediaxml.databinding.FragmentLanguageBinding
import java.util.Locale

class LanguageFragment : Fragment() {

    private var _binding: FragmentLanguageBinding? = null
    private val binding get() = _binding!!

    override fun onCreateView(
        inflater: LayoutInflater,
        container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View {
```

```
        _binding = FragmentLanguageBinding.inflate(inflater, container, false)
        return binding.root
    }

    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
        super.onViewCreated(view, savedInstanceState)

        val prefs = requireContext().getSharedPreferences("settings", Context.MODE_PRIVATE)
        val currentLang = prefs.getString("language", "id")

        updateCheckmarks(currentLang)

        binding.cardIndonesia.setOnClickListener {
            prefs.edit().putString("language", "id").apply()
            setLocale("id")
        }

        binding.cardEnglish.setOnClickListener {
            prefs.edit().putString("language", "en").apply()
            setLocale("en")
        }
    }

    private fun updateCheckmarks(lang: String?) {
        if (lang == "id") {
            binding.checkIndonesia.visibility = View.VISIBLE
            binding.checkEnglish.visibility = View.GONE
        } else {
            binding.checkEnglish.visibility = View.VISIBLE
            binding.checkIndonesia.visibility = View.GONE
        }
    }

    private fun setLocale(lang: String) {
        val locale = Locale(lang)
        Locale.setDefault(locale)
        val config = Configuration(resources.configuration)
        config.setLocale(locale)

requireActivity().resources.updateConfiguration(config, requireActivity().resources.displayMetrics)
        requireActivity().recreate()
    }
```

```

    override fun onDestroyView() {
        super.onDestroyView()
        _binding = null
    }
}
```

SongAdapter.kt

Tabel 5. Source Code SongAdapter.kt

```
package com.example.songpediaxml

import android.content.Context
import android.view.LayoutInflater
import android.view.ViewGroup
import androidx.recyclerview.widget.RecyclerView
import com.bumptech.glide.Glide
import com.bumptech.glide.load.resource.bitmap.RoundedCorners
import com.example.songpediaxml.databinding.ItemSongBinding

class SongAdapter(
    private var songs: List<Song>,
    private val onDetailClick: (Song) -> Unit,
    private val onSpotifyClick: (Song) -> Unit
) : RecyclerView.Adapter<SongAdapter.SongViewHolder>() {

    inner class SongViewHolder(val binding: ItemSongBinding) : RecyclerView.ViewHolder(binding.root)

    override fun onCreateViewHolder(parent: ViewGroup, viewType: Int): SongViewHolder {
        val binding = ItemSongBinding.inflate(LayoutInflater.from(parent.context), parent, false)
        return SongViewHolder(binding)
    }

    override fun onBindViewHolder(holder: SongViewHolder, position: Int) {
        val song = songs[position]
        val context = holder.binding.root.context

        holder.binding.tvTitle.text = song.title
        holder.binding.tvArtist.text = song.artist
        holder.binding.tvYear.text = song.year.toString()
```

```

        val prefs = context.getSharedPreferences("settings", Context.MODE_PRIVATE)
        val lang = prefs.getString("language", "id")
        if (lang == "id") {
            holder.binding.tvDescription.text = song.descriptionId
        } else {
            holder.binding.tvDescription.text = song.descriptionEn
        }

        Glide.with(context)
            .load(song.imageRes)
            .transform(RoundedCorners(24))
            .into(holder.binding.imgAlbum)

        holder.binding.btnSpotify.setOnClickListener {
            onSpotifyClick(song)
        }

        holder.binding.btnDetail.setOnClickListener {
            onDetailClick(song)
        }
    }

    override fun getItemCount() = songs.size
}
```

CarouselAdapter.kt

Tabel 6. Source Code CarouselAdapter.kt

```
package com.example.songpediaxml

import android.view.LayoutInflater
import android.view.ViewGroup
import androidx.recyclerview.widget.RecyclerView
import com.bumptech.glide.Glide
import com.bumptech.glide.load.resource.bitmap.RoundedCorners
import com.example.songpediaxml.databinding.ItemCarouselBinding

class CarouselAdapter(
    private val songs: List<Song>
) : RecyclerView.Adapter<CarouselAdapter.CarouselViewHolder>() {

```

```
    inner class CarouselViewHolder(val binding: ItemCarouselBinding) : RecyclerView.ViewHolder(binding.root)

    override fun onCreateViewHolder(parent: ViewGroup, viewType: Int): CarouselViewHolder {
        val binding = ItemCarouselBinding.inflate(LayoutInflater.from(parent.context), parent, false)
        return CarouselViewHolder(binding)
    }

    override fun onBindViewHolder(holder: CarouselViewHolder, position: Int) {
        val song = songs[position]
        Glide.with(holder.binding.root.context)
            .load(song.imageRes)
            .transform(RoundedCorners(32))
            .into(holder.binding.imgCarousel)
    }

    override fun getItemCount() = songs.size
}
```

SongViewModel.kt

Tabel 7. Source Code SongViewModel.kt

```
package com.example.songpediaxml

import android.app.Application
import androidx.lifecycle.AndroidViewModel
import androidx.lifecycle.LiveData
import androidx.lifecycle.MutableLiveData

class SongViewModel(application: Application) : AndroidViewModel(application) {

    private val _songs = MutableLiveData<List<Song>>()
    val songs: LiveData<List<Song>> = _songs

    private val _selectedSong = MutableLiveData<Song>()
    val selectedSong: LiveData<Song> = _selectedSong

    init {
        _songs.value = SongData.getSongs(application)
    }

```

```
    fun selectSong(song: Song) {
        _selectedSong.value = song
    }
}
```

Song.kt

Tabel 8. Source Code Song.kt

```
package com.example.songpediaxml

data class Song(
    val id: Int,
    val title: String,
    val artist: String,
    val album: String,
    val year: Int,
    val genre: String,
    val descriptionEn: String,
    val descriptionId: String,
    val imageRes: Int,
    val spotifyUrl: String
)
```

SongData.kt

Tabel 9. Source Code SongData.kt

```
package com.example.songpediaxml

import android.content.Context

object SongData {
    fun getSongs(context: Context): List<Song> {
        return listOf(
            Song(
                id = 1,
                title = "Should Have Known Better",
                artist = "Sufjan Stevens",
                album = "Carrie & Lowell",
                year = 2015,
                genre = "Folk",
                descriptionEn = "A deeply personal song about grief and guilt following the death of his estranged mother. It shifts from sorrow into quiet hope.",
                descriptionId = "Lagu yang sangat personal tentang duka dan rasa bersalah setelah kematian ibunya yang
```

```
telah lama terpisah. Perlahan bergerak dari kesedihan menuju harapan yang tenang.",
                imageRes = R.drawable.albumcarrieandlowell,
                spotifyUrl = "https://open.spotify.com/track/1iSI5NeLqGoTsvTk7PA5pY"
            ),
            Song(
                id = 2,
                title = "How to Disappear",
                artist = "Lana Del Rey",
                album = "Norman Fucking Rockwell!",
                year = 2019,
                genre = "Dream Pop",
                descriptionEn = "A reflective track about emotionally unavailable past lovers and learning to let go, ending with a sense of peace and self-assurance.",
                descriptionId = "Lagu reflektif tentang mantan kekasih yang tidak hadir secara emosional, dan proses belajar melepaskan, diakhiri dengan rasa damai pada diri sendiri.",
                imageRes = R.drawable.albumnormanfuckingrockwell,
                spotifyUrl = "https://open.spotify.com/track/4qUtC2BwFC154Ha8YQRrkk"
            ),
            Song(
                id = 3,
                title = "Like Real People Do",
                artist = "Hozier",
                album = "From Eden EP",
                year = 2014,
                genre = "Indie Folk",
                descriptionEn = "Inspired by Irish poet Seamus Heaney, the song is about two imperfect people choosing simple, honest intimacy over their complicated pasts.",
                descriptionId = "Terinspirasi dari puisi Seamus Heaney, lagu ini tentang dua orang yang tidak sempurna memilih keintiman sederhana dan jujur daripada masa lalu mereka yang rumit.",
                imageRes = R.drawable.albumfromeden,
                spotifyUrl = "https://open.spotify.com/track/4LGJ2pLDvTRnul3EcZoYkX"
            ),
            Song(
                id = 4,
```

```
                title = "The Moon Will Sing",
                artist = "The Crane Wives",
                album = "Coyote Stories",
                year = 2015,
                genre = "Indie Folk",
                descriptionEn = "A bittersweet song about losing one's identity in a relationship, using the moon and sun as a metaphor for emotional dependency.",
                descriptionId = "Lagu bittersweet tentang kehilangan jati diri dalam sebuah hubungan, menggunakan bulan dan matahari sebagai metafora untuk ketergantungan emosional.",
                imageRes = R.drawable.albumherons,
                spotifyUrl = "https://open.spotify.com/track/4v0Zqmuhlbt5nCj712Y26n"
            ),
            Song(
                id = 5,
                title = "Duvet",
                artist = "Bôa",
                album = "Twilight",
                year = 2001,
                genre = "Alternative Rock",
                descriptionEn = "Originally about communication and relying on others, the song became iconic as the opening theme of the anime Serial Experiments Lain.",
                descriptionId = "Awalnya tentang komunikasi dan ketergantungan pada orang lain, lagu ini menjadi ikonik sebagai lagu pembuka anime Serial Experiments Lain.",
                imageRes = R.drawable.albumtwilight,
                spotifyUrl = "https://open.spotify.com/track/42qNWdLKCI41S4uzfamhFM"
            ),
            Song(
                id = 6,
                title = "Please Be Rude",
                artist = "Gigi Perez",
                album = "Please Be Rude",
                year = 2024,
                genre = "Indie Pop",
                descriptionEn = "A soft and vulnerable song about the longing for genuine intimacy and the desire to be seen honestly in a romantic relationship.",
                descriptionId = "Lagu yang lembut dan penuh kerentanan tentang kerinduan akan keintiman sejati dan keinginan untuk dilihat apa adanya dalam sebuah hubungan romantis.",
```

```
                imageRes = R.drawable.albumpleaseberude,
                spotifyUrl                                  =
"https://open.spotify.com/track/2ORD5RHyZI9LGibgqcwr3v"
            ),
            Song(
                id = 7,
                title = "Buzzcut Season",
                artist = "Lorde",
                album = "Pure Heroine",
                year = 2013,
                genre = "Electropop",
                descriptionEn = "A deceptively calm song
about teenage escapism and blissful ignorance, while quietly
acknowledging the darkness of the outside world.",
                descriptionId = "Lagu yang terdengar tenang
namun menyimpan makna dalam tentang pelarian remaja dan
kebahagiaan yang naif, sambil diam-diam mengakui kegelapan
dunia luar.",
                imageRes = R.drawable.albumpureheroine,
                spotifyUrl                                  =
"https://open.spotify.com/track/51QEyJI5M7uyd8DOh9tqQY"
            )
        )
    }
}
```

activity_main.xml

Tabel 10. Source Code activity_main.kt

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout

xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="#12121E">

    <androidx.fragment.app.FragmentContainerView
        android:id="@+id/navHostFragment"

android:name="androidx.navigation.fragment.NavHostFragment"
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        app:defaultNavHost="true"
        app:navGraph="@navigation/nav_graph"
```

```
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintBottom_toBottomOf="parent" />

</androidx.constraintlayout.widget.ConstraintLayout>
```

fragment_home.xml

Tabel 11. Source Code fragment_home.kt

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout

xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:background="#FFFFFF">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="horizontal"
        android:paddingHorizontal="16dp"
        android:paddingTop="16dp"
        android:paddingBottom="8dp"
        android:background="#8B6343"
        android:gravity="center_vertical">

        <TextView
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_weight="1"
            android:text="@string/home_title"
            android:textColor="#FFFFFF"
            android:textSize="24sp"
            android:textStyle="bold" />

        <com.google.android.material.button.MaterialButton
            android:id="@+id/btnLanguage"

style="@style/Widget.MaterialComponents.Button.TextButton"
            android:layout_width="34dp"
            android:layout_height="48dp"
            app:icon="@drawable/ic_language"
            app:iconTint="#FFFFFF"
            app:iconSize="28dp"
```

```
            app:backgroundTint="@android:color/transparent"
            app:strokeWidth="0dp"
            android:contentDescription="Change Language" />

    </LinearLayout>

    <androidx.core.widget.NestedScrollView
        android:layout_width="match_parent"
        android:layout_height="match_parent">

        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:orientation="vertical">

            <TextView
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:text="@string/label_highlighted"
                android:textColor="#3E2723"
                android:textSize="16sp"
                android:textStyle="bold"
                android:paddingHorizontal="16dp"
                android:paddingTop="8dp"
                android:paddingBottom="4dp" />

            <androidx.recyclerview.widget.RecyclerView
                android:id="@+id/rvCarousel"
                android:layout_width="match_parent"
                android:layout_height="180dp"
                android:orientation="horizontal"
                android:clipToPadding="false"
                android:paddingHorizontal="16dp"
                android:nestedScrollingEnabled="false"
app:layoutManager="androidx.recyclerview.widget.LinearLayoutManager" />

            <TextView
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:text="@string/label_all_songs"
                android:textColor="#3E2723"
                android:textSize="16sp"
                android:textStyle="bold"
                android:paddingHorizontal="16dp"
                android:paddingTop="16dp"
```

```
                android:paddingBottom="4dp" />

            <androidx.recyclerview.widget.RecyclerView
                android:id="@+id/rvSongs"
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:paddingBottom="16dp"
                android:nestedScrollingEnabled="false" />

        </LinearLayout>

    </androidx.core.widget.NestedScrollView>

</LinearLayout>
```

fragment_detail.xml

Tabel 12. Source Code fragment_detail.kt

```
<?xml version="1.0" encoding="utf-8"?>
<ScrollView

xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="#FFFFFF">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical">

        <ImageView
            android:id="@+id/imgDetailAlbum"
            android:layout_width="match_parent"
            android:layout_height="400dp"
            android:scaleType="centerCrop"
            android:layout_margin="16dp"

android:background="@drawable/rounded_image_bg"
            android:clipToOutline="true" />

        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:orientation="vertical"
            android:padding="20dp">
```

```
27
28              <LinearLayout
29                  android:layout_width="match_parent"
30                  android:layout_height="wrap_content"
31                  android:orientation="horizontal"
32                  android:gravity="center_vertical">
33
34                  <TextView
35                      android:id="@+id/tvDetailTitle"
36                      android:layout_width="0dp"
37                      android:layout_height="wrap_content"
38                      android:layout_weight="1"
39                      android:textColor="#3E2723"
40                      android:textSize="22sp"
41                      android:textStyle="bold" />
42
43                  <TextView
44                      android:id="@+id/tvDetailYear"
45                      android:layout_width="wrap_content"
46                      android:layout_height="wrap_content"
47                      android:textColor="#6D4B3F"
48                      android:textSize="16sp"
49                      android:layout_marginStart="8dp" />
50
51              </LinearLayout>
52
53              <TextView
54                  android:id="@+id/tvDetailArtist"
55                  android:layout_width="wrap_content"
56                  android:layout_height="wrap_content"
57                  android:textColor="#6D4B3F"
58                  android:textSize="16sp"
59                  android:layout_marginTop="4dp" />
60
61              <View
62                  android:layout_width="match_parent"
63                  android:layout_height="1dp"
64                  android:background="#3E2723"
65                  android:layout_marginVertical="16dp" />
66
67              <TextView
68                  android:layout_width="wrap_content"
69                  android:layout_height="wrap_content"
70                  android:text="@string/label_album"
71                  android:textColor="#3E2723"
72                  android:textSize="15sp"
73                  android:textAllCaps="true"
```

```
74                  android:letterSpacing="0.1" />
75
76              <TextView
77                  android:id="@+id/tvDetailAlbum"
78                  android:layout_width="wrap_content"
79                  android:layout_height="wrap_content"
80                  android:textColor="#6D4B3F"
81                  android:textSize="14sp"
82                  android:layout_marginTop="2dp" />
83
84              <TextView
85                  android:layout_width="wrap_content"
86                  android:layout_height="wrap_content"
87                  android:text="@string/label_genre"
88                  android:textColor="#3E2723"
89                  android:textSize="15sp"
90                  android:textAllCaps="true"
91                  android:letterSpacing="0.1"
92                  android:layout_marginTop="12dp" />
93
94              <TextView
95                  android:id="@+id/tvDetailGenre"
96                  android:layout_width="wrap_content"
97                  android:layout_height="wrap_content"
98                  android:textColor="#6D4B3F"
99                  android:textSize="14sp"
100                 android:layout_marginTop="2dp" />
101
102             <TextView
103                 android:id="@+id/tvDescriptionLabel"
104                 android:layout_width="wrap_content"
105                 android:layout_height="wrap_content"
106                 android:textColor="#3E2723"
107                 android:textSize="15sp"
108                 android:textAllCaps="true"
109                 android:letterSpacing="0.1"
110                 android:layout_marginTop="12dp" />
111
112             <TextView
113                 android:id="@+id/tvDetailDescription"
114                 android:layout_width="match_parent"
115                 android:layout_height="wrap_content"
116                 android:textColor="#6D4B3F"
117                 android:textSize="14sp"
118                 android:lineSpacingExtra="4dp"
119                 android:layout_marginTop="2dp" />
120
```

```
        </LinearLayout>

    </LinearLayout>

</ScrollView>
```

fragment_language.xml

Tabel 13. Source Code fragment_language.kt

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout

xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:background="#FFFFFF"
    android:gravity="center"
    android:padding="32dp">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="@string/label_choose_language"
        android:textColor="#3E2723"
        android:textSize="22sp"
        android:textStyle="bold"
        android:layout_marginBottom="32dp" />

    <com.google.android.material.card.MaterialCardView
        android:id="@+id/cardIndonesia"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        app:cardCornerRadius="16dp"
        app:cardBackgroundColor="#EDE0D4"
        app:cardElevation="4dp"
        app:strokeColor="#EDE0D4"
        app:strokeWidth="2dp"
        android:layout_marginBottom="16dp">

        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:orientation="horizontal"
            android:padding="20dp"
```

```
            android:gravity="center_vertical">

            <TextView
                android:layout_width="0dp"
                android:layout_height="wrap_content"
                android:layout_weight="1"
                android:text="@string/label_indonesia"
                android:textColor="#3E2723"
                android:textSize="16sp" />

            <ImageView
                android:id="@+id/checkIndonesia"
                android:layout_width="24dp"
                android:layout_height="24dp"
android:src="@android:drawable/checkbox_on_background"
                android:visibility="gone" />

        </LinearLayout>

    </com.google.android.material.card.MaterialCardView>

    <com.google.android.material.card.MaterialCardView
        android:id="@+id/cardEnglish"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        app:cardCornerRadius="16dp"
        app:cardBackgroundColor="#EDE0D4"
        app:cardElevation="4dp"
        app:strokeColor="#EDE0D4"
        app:strokeWidth="2dp">

        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:orientation="horizontal"
            android:padding="20dp"
            android:gravity="center_vertical">

            <TextView
                android:layout_width="0dp"
                android:layout_height="wrap_content"
                android:layout_weight="1"
                android:text="@string/label_english"
                android:textColor="#3E2723"
                android:textSize="16sp" />

```

```
            <ImageView
                android:id="@+id/checkEnglish"
                android:layout_width="24dp"
                android:layout_height="24dp"
android:src="@android:drawable/checkbox_on_background"
                android:visibility="gone" />

        </LinearLayout>

    </com.google.android.material.card.MaterialCardView>

</LinearLayout>
```

item_song.xml

Tabel 14. Source Code item_song.kt

```
<?xml version="1.0" encoding="utf-8"?>
<com.google.android.material.card.MaterialCardView
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginHorizontal="16dp"
    android:layout_marginVertical="8dp"
    app:cardCornerRadius="16dp"
    app:cardBackgroundColor="#F3F6EEE5"
    app:cardElevation="4dp">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="horizontal"
        android:padding="12dp">

        <ImageView
            android:id="@+id/imgAlbum"
            android:layout_width="110dp"
            android:layout_height="110dp"
            android:scaleType="centerCrop"
            android:clipToOutline="true"
android:background="@drawable/rounded_image_bg" />

        <LinearLayout
```

```
28              android:layout_width="match_parent"
29              android:layout_height="wrap_content"
30              android:orientation="vertical"
31              android:layout_marginStart="12dp">
32
33              <LinearLayout
34                  android:layout_width="match_parent"
35                  android:layout_height="wrap_content"
36                  android:orientation="horizontal">
37
38                  <TextView
39                      android:id="@+id/tvTitle"
40                      android:layout_width="0dp"
41                      android:layout_height="wrap_content"
42                      android:layout_weight="1"
43                      android:textColor="#3E2723"
44                      android:textSize="14sp"
45                      android:textStyle="bold"
46                      android:maxLines="2"
47                      android:ellipsize="end" />
48
49                  <TextView
50                      android:id="@+id/tvYear"
51                      android:layout_width="wrap_content"
52                      android:layout_height="wrap_content"
53                      android:textColor="#6D4B3F"
54                      android:textSize="12sp"
55                      android:layout_marginStart="8dp" />
56
57              </LinearLayout>
58
59              <TextView
60                  android:id="@+id/tvArtist"
61                  android:layout_width="wrap_content"
62                  android:layout_height="wrap_content"
63                  android:textColor="#3E2723"
64                  android:textSize="12sp"
65                  android:layout_marginTop="2dp" />
66
67              <TextView
68                  android:id="@+id/tvDescription"
69                  android:layout_width="match_parent"
70                  android:layout_height="wrap_content"
71                  android:textColor="#6D4B3F"
72                  android:textSize="11sp"
73                  android:maxLines="2"
74                  android:ellipsize="end"
```

```
                android:layout_marginTop="4dp" />

            <LinearLayout
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:orientation="horizontal"
                android:layout_marginTop="8dp"
                android:gravity="end">


<com.google.android.material.button.MaterialButton
                    android:id="@+id/btnSpotify"
                    android:layout_width="wrap_content"
                    android:layout_height="40dp"
                    android:text="@string/btn_spotify"
                    android:textSize="12sp"
                    android:textColor="#FFFFFF"
                    app:backgroundTint="#8B6343"
                    app:cornerRadius="20dp"
                    android:paddingHorizontal="16dp"
                    android:textAllCaps="false"
style="@style/Widget.MaterialComponents.Button" />


<com.google.android.material.button.MaterialButton
                    android:id="@+id/btnDetail"
                    android:layout_width="wrap_content"
                    android:layout_height="40dp"
                    android:text="@string/btn_detail"
                    android:textSize="12sp"
                    android:textColor="#FFFFFF"
                    app:backgroundTint="#8B6343"
                    app:cornerRadius="20dp"
                    android:paddingHorizontal="16dp"
                    android:layout_marginStart="8dp"
                    android:textAllCaps="false"
style="@style/Widget.MaterialComponents.Button" />

            </LinearLayout>

        </LinearLayout>

    </LinearLayout>

</com.google.android.material.card.MaterialCardView>
```

item_carousel.xml

Tabel 15. Source Code item_carousel.kt

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="150dp"
    android:layout_height="150dp"
    android:orientation="vertical"
    android:layout_marginEnd="12dp">

    <ImageView
        android:id="@+id/imgCarousel"
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:scaleType="centerCrop"
        android:background="@drawable/rounded_image_bg"
        android:clipToOutline="true" />
</LinearLayout>
```

nav_graph.xml

Tabel 16. Source Code nav_graph.kt

```
<?xml version="1.0" encoding="utf-8"?>
<navigation
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:id="@+id/nav_graph"
    app:startDestination="@id/homeFragment">

    <fragment
        android:id="@+id/homeFragment"

android:name="com.example.songpediaxml.HomeFragment"
        android:label="Home">
        <action
            android:id="@+id/action_home_to_detail"
            app:destination="@id/detailFragment" />
        <action
            android:id="@+id/action_home_to_language"
            app:destination="@id/languageFragment" />
    </fragment>

    <fragment
        android:id="@+id/detailFragment"
```

```
21
      android:name="com.example.songpediaxml.DetailFragment"
22          android:label="Detail" />
23
24      <fragment
25          android:id="@+id/languageFragment"
26
      android:name="com.example.songpediaxml.LanguageFragment"
27          android:label="Language" />
28
29    </navigation>
```

strings.xml (in)

Tabel 17. Source Code strings.kt (in)

```
1     <resources>
2         <string name="app_name">SongPedia</string>
3         <string name="home_title">SongPedia</string>
4         <string name="label_album">Album</string>
5         <string name="label_genre">Genre</string>
6         <string name="label_description">Deskripsi</string>
7         <string             name="label_choose_language">Pilih
      Bahasa</string>
8         <string                 name="label_indonesia">Bahasa
      Indonesia</string>
9         <string name="label_english">English</string>
10        <string name="btn_spotify">Spotify</string>
11        <string name="btn_detail">Detail</string>
12        <string name="label_highlighted">Lagu Pilihan</string>
13        <string name="label_all_songs">Semua Lagu</string>
14    </resources>
```

strings.xml

Tabel 18. Source Code strings.kt

```
<resources>
    <string name="app_name">SongPedia</string>
    <string name="home_title">SongPedia</string>
    <string name="label_album">Album</string>
    <string name="label_genre">Genre</string>
    <string name="label_description">Description</string>
    <string name="label_choose_language">Choose Language</string>
    <string name="label_indonesia">Bahasa Indonesia</string>
    <string name="label_english">English</string>
    <string name="btn_spotify">Spotify</string>
    <string name="btn_detail">Detail</string>
    <string name="label_highlighted">Highlighted Songs</string>
    <string name="label_all_songs">All Songs</string>
</resources>
```

themes.xml

Tabel 19. Source Code themes.kt

```
<resources xmlns:tools="http://schemas.android.com/tools">
    <style name="Base.Theme.SongPediaXML" parent="Theme.Material3.DayNight.NoActionBar">
    </style>

    <style name="Theme.SongPediaXML" parent="Theme.MaterialComponents.Light.NoActionBar">
        <item name="colorPrimary">#3E2723</item>
        <item name="android:statusBarColor">#8B6343</item>
        <item name="colorSurface">#8B6343</item>
    </style>
</resources>
```

colors.xml

Tabel 20. Source Code colosr.kt

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <color name="black">#FF000000</color>
    <color name="white">#FFFFFFFF</color>
</resources>
```

COMPOSE

HomeScreen.kt

Tabel 21. Source Code HomeScreen.kt

```
package com.example.songpediacompose

import android.content.Intent
import android.net.Uri
import androidx.compose.foundation.background
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.PaddingValues
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.layout.size
import androidx.compose.foundation.layout.width
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.lazy.LazyRow
import androidx.compose.foundation.lazy.items
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.outlined.Language
import androidx.compose.material3.Button
import androidx.compose.material3.ButtonDefaults
import androidx.compose.material3.Card
import androidx.compose.material3.CardDefaults
import androidx.compose.material3.Icon
import androidx.compose.material3.IconButton
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.runtime.getValue
import androidx.compose.runtime.livedata.observeAsState
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.draw.clip
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.layout.ContentScale
import androidx.compose.ui.platform.LocalContext
import androidx.compose.ui.res.stringResource
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.text.style.TextOverflow
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
```

```
import coil.compose.AsyncImage
import com.example.songpediacompose.ui.theme.TextBrown
import com.example.songpediacompose.ui.theme.TextDark
import com.example.songpediacompose.ui.theme.White
import com.example.songpediacompose.viewmodel.SongViewModel

val HeaderColor = Color(0xFF8B6343)
val ButtonBrown = Color(0xFF8B6343)
val CardBg = Color(0xFFF0EAE2)

@Composable
fun HomeScreen(
    viewModel: SongViewModel,
    onDetailClick: (Song) -> Unit,
    onLanguageClick: () -> Unit
) {
    val context = LocalContext.current
    val songs by viewModel.songs.observeAsState(emptyList())
    val prefs = context.getSharedPreferences("settings", android.content.Context.MODE_PRIVATE)
    val lang = prefs.getString("language", "id") ?: "id"

    Column(
        modifier = Modifier
            .fillMaxSize()
            .background(Color.White)
    ) {

        Row(
            modifier = Modifier
                .fillMaxWidth()
                .background(HeaderColor)
                .padding(start = 16.dp, end = 4.dp, top = 16.dp, bottom = 8.dp),
            verticalAlignment = Alignment.CenterVertically,
            horizontalArrangement = Arrangement.SpaceBetween
        ) {
            Text(
                text = stringResource(R.string.home_title),
                fontSize = 24.sp,
                fontWeight = FontWeight.Bold,
                color = White
            )

            IconButton(
```

```
                onClick = onLanguageClick,
                modifier = Modifier.size(34.dp)
            ) {
                Icon(
                    imageVector = Icons.Outlined.Language,
                    contentDescription = "Change Language",
                    tint = White,
                    modifier = Modifier.size(28.dp)
                )
            }
        }

        LazyColumn(
            modifier = Modifier
                .fillMaxSize()
                .background(Color.White),
            contentPadding = PaddingValues(bottom = 16.dp)
        ) {
            item {
                Text(
                    text = stringResource(R.string.label_highlighted),
                    fontSize = 16.sp,
                    fontWeight = FontWeight.Bold,
                    color = TextDark,
                    modifier = Modifier.padding(
                        start = 16.dp, end = 16.dp,
                        top = 8.dp, bottom = 4.dp
                    )
                )

                LazyRow(
                    modifier = Modifier.height(180.dp),
                    contentPadding = PaddingValues(horizontal = 16.dp),
                    horizontalArrangement = Arrangement.spacedBy(12.dp)
                ) {
                    items(songs) { song ->
                        AsyncImage(
                            model = song.imageRes,
                            contentDescription = song.title,
                            contentScale = ContentScale.Crop,
                            modifier = Modifier
                                .size(150.dp)
```

```
.clip(RoundedCornerShape(12.dp))
                        )
                    }
                }

                Text(
                    text = stringResource(R.string.label_all_songs),
                    fontSize = 16.sp,
                    fontWeight = FontWeight.Bold,
                    color = TextDark,
                    modifier = Modifier.padding(
                        start = 16.dp, end = 16.dp,
                        top = 16.dp, bottom = 4.dp
                    )
                )
            }

            items(songs) { song ->
                SongItem(
                    song = song,
                    lang = lang,
                    onDetailClick = onDetailClick,
                    onSpotifyClick = {
                        val intent = Intent(Intent.ACTION_VIEW, Uri.parse(song.spotifyUrl))
                        context.startActivity(intent)
                    }
                )
            }
        }
    }
}

@Composable
fun SongItem(
    song: Song,
    lang: String,
    onDetailClick: (Song) -> Unit,
    onSpotifyClick: () -> Unit
) {
    Card(
        modifier = Modifier
            .fillMaxWidth()
            .padding(horizontal = 16.dp, vertical = 8.dp),
        shape = RoundedCornerShape(16.dp),
```

```
173         colors = CardDefaults.cardColors(containerColor =
    CardBg),
174         elevation =
    CardDefaults.cardElevation(defaultElevation = 4.dp)
175     ) {
176         Row(
177             modifier = Modifier.padding(12.dp),
178             verticalAlignment = Alignment.Top
179         ) {
180             AsyncImage(
181                 model = song.imageRes,
182                 contentDescription = song.title,
183                 contentScale = ContentScale.Crop,
184                 modifier = Modifier
185                     .size(110.dp)
186                     .clip(RoundedCornerShape(12.dp))
187             )
188             Spacer(modifier = Modifier.width(12.dp))
189             Column(modifier = Modifier.weight(1f)) {
190                 Row(
191                     modifier = Modifier.fillMaxWidth(),
192                     horizontalArrangement =
    Arrangement.SpaceBetween
193                 ) {
194                     Text(
195                         text = song.title,
196                         fontWeight = FontWeight.Bold,
197                         fontSize = 14.sp,
198                         color = TextDark,
199                         maxLines = 2,
200                         overflow = TextOverflow.Ellipsis,
201                         modifier = Modifier.weight(1f)
202                     )
203                     Text(
204                         text = song.year.toString(),
205                         fontSize = 12.sp,
206                         color = TextBrown,
207                         modifier = Modifier.padding(start =
    8.dp)
208                     )
209                 }
210                 Text(
211                     text = song.artist,
212                     fontSize = 12.sp,
213                     color = TextDark,
214                     modifier = Modifier.padding(top = 2.dp)
215                 )
```

```
216                 Spacer(modifier = Modifier.height(4.dp))
217                 Text(
218                     text = if (lang == "id")
    song.descriptionId else song.descriptionEn,
219                     fontSize = 11.sp,
220                     color = TextBrown,
221                     maxLines = 2,
222                     overflow = TextOverflow.Ellipsis
223                 )
224                 Spacer(modifier = Modifier.height(8.dp))
225
226                 Row(
227                     modifier = Modifier.fillMaxWidth(),
228                     horizontalArrangement = Arrangement.End
229                 ) {
230                     Button(
231                         onClick = onSpotifyClick,
232                         colors =
    ButtonDefaults.buttonColors(containerColor = ButtonBrown),
233                         shape = RoundedCornerShape(20.dp),
234                         contentPadding =
    PaddingValues(horizontal = 16.dp, vertical = 8.dp)
235                     ) {
236                         Text(
237                             text =
    stringResource(R.string.btn_spotify),
238                             color = White,
239                             fontSize = 12.sp
240                         )
241                     }
242                     Spacer(modifier = Modifier.width(8.dp))
243                     Button(
244                         onClick = { onDetailClick(song) },
245                         colors =
    ButtonDefaults.buttonColors(containerColor = ButtonBrown),
246                         shape = RoundedCornerShape(20.dp),
247                         contentPadding =
    PaddingValues(horizontal = 16.dp, vertical = 8.dp)
248                     ) {
249                         Text(
250                             text =
    stringResource(R.string.btn_detail),
251                             color = White,
252                             fontSize = 12.sp
253                         )
254                     }
255                 }
```

```
256                 }
257             }
258         }
259 }
```

DetailScreen.kt

Tabel 22. Source Code DetailScreen.kt

```
1   package com.example.songpediacompose
2
3   import androidx.compose.foundation.Canvas
4   import androidx.compose.foundation.background
5   import androidx.compose.foundation.layout.Column
6   import androidx.compose.foundation.layout.Row
7   import androidx.compose.foundation.layout.Spacer
8   import androidx.compose.foundation.layout.fillMaxSize
9   import androidx.compose.foundation.layout.fillMaxWidth
10  import androidx.compose.foundation.layout.height
11  import androidx.compose.foundation.layout.padding
12  import androidx.compose.foundation.rememberScrollState
13  import androidx.compose.foundation.shape.RoundedCornerShape
14  import androidx.compose.foundation.verticalScroll
15  import androidx.compose.material3.Text
16  import androidx.compose.runtime.Composable
17  import androidx.compose.runtime.getValue
18  import androidx.compose.runtime.livedata.observeAsState
19  import androidx.compose.ui.Modifier
20  import androidx.compose.ui.draw.clip
21  import androidx.compose.ui.graphics.Color
22  import androidx.compose.ui.layout.ContentScale
23  import androidx.compose.ui.platform.LocalContext
24  import androidx.compose.ui.res.stringResource
25  import androidx.compose.ui.text.font.FontWeight
26  import androidx.compose.ui.unit.dp
27  import androidx.compose.ui.unit.sp
28  import coil.compose.AsyncImage
29  import com.example.songpediacompose.ui.theme.TextDark
30  import com.example.songpediacompose.viewmodel.SongViewModel
31
32  val DetailTextBrown = Color(0xFF6D4B3F)
33
34  @Composable
35  fun DetailScreen(
36      viewModel: SongViewModel,
37      onBack: () -> Unit
38  ) {
39      val context = LocalContext.current
```

```
    val song: Song? by viewModel.selectedSong.observeAsState(initial = null)
    val prefs = context.getSharedPreferences("settings", android.content.Context.MODE_PRIVATE)
    val lang = prefs.getString("language", "id") ?: "id"

    if (song != null) {
        val s = song!!
        Column(
            modifier = Modifier
                .fillMaxSize()
                .background(Color.White)
                .verticalScroll(rememberScrollState())
        ) {


            AsyncImage(
                model = s.imageRes,
                contentDescription = s.title,
                contentScale = ContentScale.Crop,
                modifier = Modifier
                    .fillMaxWidth()
                    .height(400.dp)
                    .padding(all = 16.dp)
                    .clip(RoundedCornerShape(12.dp))
            )


            Column(modifier = Modifier.padding(horizontal = 20.dp, vertical = 0.dp)) {


                Row(modifier = Modifier.fillMaxWidth()) {
                    Text(
                        text = s.title,
                        fontSize = 22.sp,
                        fontWeight = FontWeight.Bold,
                        color = TextDark,
                        modifier = Modifier.weight(1f)
                    )
                    Text(
                        text = s.year.toString(),
                        fontSize = 16.sp,
                        color = DetailTextBrown,
                        modifier = Modifier.padding(start = 8.dp)
                    )
```

```
                }


                Text(
                    text = s.artist,
                    fontSize = 16.sp,
                    color = DetailTextBrown,
                    modifier = Modifier.padding(top = 4.dp)
                )


                Spacer(modifier = Modifier.height(16.dp))
                Canvas(
                    modifier = Modifier
                        .fillMaxWidth()
                        .height(1.dp)
                ) {
                    drawRect(color = Color(0xFF3E2723))
                }
                Spacer(modifier = Modifier.height(16.dp))


                Text(
                    text = stringResource(R.string.label_album).uppercase(),
                    fontSize = 15.sp,
                    fontWeight = FontWeight.Normal,
                    color = TextDark,
                    letterSpacing = 0.1.sp
                )
                Text(
                    text = s.album,
                    fontSize = 14.sp,
                    color = DetailTextBrown,
                    modifier = Modifier.padding(top = 2.dp)
                )

                Spacer(modifier = Modifier.height(12.dp))


                Text(
                    text = stringResource(R.string.label_genre).uppercase(),
                    fontSize = 15.sp,
                    fontWeight = FontWeight.Normal,
                    color = TextDark,
                    letterSpacing = 0.1.sp
```

```
                )
                Text(
                    text = s.genre,
                    fontSize = 14.sp,
                    color = DetailTextBrown,
                    modifier = Modifier.padding(top = 2.dp)
                )

                Spacer(modifier = Modifier.height(12.dp))


                Text(
                    text = stringResource(R.string.label_description).uppercase(),
                    fontSize = 15.sp,
                    fontWeight = FontWeight.Normal,
                    color = TextDark,
                    letterSpacing = 0.1.sp
                )
                Text(
                    text = if (lang == "id") s.descriptionId else s.descriptionEn,
                    fontSize = 14.sp,
                    color = DetailTextBrown,
                    lineHeight = 20.sp,
                    modifier = Modifier.padding(top = 2.dp, bottom = 20.dp)
                )
            }
        }
    }
}
```

LanguageScreen.kt

Tabel 23. Source Code LanguageScreen.kt

```
package com.example.songpediacompose

import android.content.Context
import android.content.res.Configuration
import androidx.compose.foundation.background
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.padding
```

```
import androidx.compose.foundation.layout.size
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.CheckBox
import androidx.compose.material3.Card
import androidx.compose.material3.CardDefaults
import androidx.compose.material3.Icon
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.setValue
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.platform.LocalContext
import androidx.compose.ui.res.stringResource
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import com.example.songpediacompose.ui.theme.TextDark
import java.util.Locale


val LangCardColor = Color(0xFFEDE0D4)

@Composable
fun LanguageScreen(onBack: () -> Unit) {
    val context = LocalContext.current
    val prefs = context.getSharedPreferences("settings", Context.MODE_PRIVATE)
    var selectedLang by remember {
        mutableStateOf(prefs.getString("language", "id") ?: "id")
    }

    Column(
        modifier = Modifier
            .fillMaxSize()
            .background(Color.White)

            .padding(32.dp),
        verticalArrangement = Arrangement.Center,
        horizontalAlignment = Alignment.CenterHorizontally
    ) {

```

```
        Text(
            text = stringResource(R.string.label_choose_language),
            fontSize = 22.sp,
            fontWeight = FontWeight.Bold,
            color = TextDark,
            modifier = Modifier.padding(bottom = 32.dp)
        )


        Card(
            onClick = {
                selectedLang = "id"
                prefs.edit().putString("language", "id").apply()
                setLocale(context, "id")
            },
            modifier = Modifier
                .fillMaxWidth()
                .padding(bottom = 16.dp),
            shape = RoundedCornerShape(16.dp),
            colors = CardDefaults.cardColors(containerColor = LangCardColor),
            elevation = CardDefaults.cardElevation(defaultElevation = 4.dp)
        ) {
            Row(
                modifier = Modifier
                    .fillMaxWidth()
                    .padding(20.dp),
                horizontalArrangement = Arrangement.SpaceBetween,
                verticalAlignment = Alignment.CenterVertically
            ) {
                Text(
                    text = stringResource(R.string.label_indonesia),
                    fontSize = 16.sp,
                    color = TextDark
                )
                if (selectedLang == "id") {
                    Icon(
                        imageVector = Icons.Filled.CheckBox,
                        contentDescription = null,
                        modifier = Modifier.size(24.dp),
```

```
                        tint = TextDark
                    )
                }
            }
        }


        Card(
            onClick = {
                selectedLang = "en"
                prefs.edit().putString("language", "en").apply()
                setLocale(context, "en")
            },
            modifier = Modifier.fillMaxWidth(),
            shape = RoundedCornerShape(16.dp),
            colors = CardDefaults.cardColors(containerColor = LangCardColor),
            elevation = CardDefaults.cardElevation(defaultElevation = 4.dp)
        ) {
            Row(
                modifier = Modifier
                    .fillMaxWidth()
                    .padding(20.dp),
                horizontalArrangement = Arrangement.SpaceBetween,
                verticalAlignment = Alignment.CenterVertically
            ) {
                Text(
                    text = stringResource(R.string.label_english),
                    fontSize = 16.sp,
                    color = TextDark
                )
                if (selectedLang == "en") {
                    Icon(
                        imageVector = Icons.Filled.CheckBox,
                        contentDescription = null,
                        modifier = Modifier.size(24.dp),
                        tint = TextDark
                    )
                }
            }
        }
```

```
    }
}

fun setLocale(context: Context, lang: String) {
    val locale = Locale(lang)
    Locale.setDefault(locale)
    val config = Configuration(context.resources.configuration)
    config.setLocale(locale)
    context.resources.updateConfiguration(config, context.resources.displayMetrics)
    (context as? android.app.Activity)?.recreate()
}
```

MainActivity.kt

Tabel 24. Source Code MainActivity.kt

```
package com.example.songpediacompose

import android.content.Context
import android.content.res.Configuration
import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.activity.enableEdgeToEdge
import androidx.appcompat.app.AppCompatDelegate
import androidx.compose.foundation.layout.Box
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.statusBarsPadding
import androidx.compose.material3.Surface
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import com.example.songpediacompose.ui.theme.SongPediaTheme
import java.util.Locale

class MainActivity : ComponentActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        AppCompatDelegate.setDefaultNightMode(AppCompatDelegate.MODE_NIGHT_NO)

        val prefs = getSharedPreferences("settings", Context.MODE_PRIVATE)
        if (!prefs.contains("language")) {
            prefs.edit().putString("language", "id").apply()
        }
```

```
29          val lang = prefs.getString("language", "id") ?: "id"
30          val locale = Locale(lang)
31          Locale.setDefault(locale)
32          val config = Configuration(resources.configuration)
33          config.setLocale(locale)
34          resources.updateConfiguration(config,
  resources.displayMetrics)
35
36          enableEdgeToEdge()
37          setContent {
38              SongPediaTheme {
39                  Surface(
40                      modifier = Modifier.fillMaxSize(),
41                      color = Color(0xFF8B6343)
42                  ) {
43                      Box(modifier = Modifier.statusBarsPadding()) {
44                          NavGraph()
45                      }
46                  }
47              }
48          }
49      }
50  }
```

NavGraph.kt

Tabel 25. Source Code NavGraph.kt

```
1   package com.example.songpediacompose
2
3   import androidx.compose.runtime.Composable
4   import androidx.lifecycle.viewmodel.compose.viewModel
5   import androidx.navigation.compose.NavHost
6   import androidx.navigation.compose.composable
7   import androidx.navigation.compose.rememberNavController
8   import com.example.songpediacompose.viewmodel.SongViewModel
9
10  @Composable
11  fun NavGraph() {
12      val navController = rememberNavController()
13      val viewModel: SongViewModel = viewModel()
14
15      NavHost(navController = navController, startDestination
    = "home") {
16          composable("home") {
17              HomeScreen(
18                  viewModel = viewModel,
19                  onDetailClick = { song ->
```

```
20                      viewModel.selectSong(song)
21                      navController.navigate("detail")
22                  },
23                  onLanguageClick = {
24                      navController.navigate("language")
25                  }
26              )
27          }
28          composable("detail") {
29              DetailScreen(
30                  viewModel = viewModel,
31                  onBack = {
32                      navController.popBackStack()
33                  }
34              )
35          }
36          composable("language") {
37              LanguageScreen(
38                  onBack = {
39                      navController.popBackStack()
40                  }
41              )
42          }
43      }
44  }
```

Song.kt

Tabel 26. Source Code Song.kt

```
1   package com.example.songpediacompose
2
3   data class Song(
4       val id: Int,
5       val title: String,
6       val artist: String,
7       val album: String,
8       val year: Int,
9       val genre: String,
10      val descriptionEn: String,
11      val descriptionId: String,
12      val imageRes: Int,
13      val spotifyUrl: String
14  )
```

SongData.kt

Tabel 27. Source Code SongData.kt

```
1   package com.example.songpediacompose.data
2
3   import android.content.Context
4   import com.example.songpediacompose.R
5   import com.example.songpediacompose.Song
6
7   object SongData {
8       fun getSongs(context: Context): List<Song> {
9           return listOf(
10              Song(
11                  id = 1,
12                  title = "Should Have Known Better",
13                  artist = "Sufjan Stevens",
14                  album = "Carrie & Lowell",
15                  year = 2015,
16                  genre = "Folk",
17                  descriptionEn = "A deeply personal song
    about grief and guilt following the death of his estranged
    mother. It shifts from sorrow into quiet hope.",
18                  descriptionId = "Lagu yang sangat personal
    tentang duka dan rasa bersalah setelah kematian ibunya yang
    telah lama terpisah. Perlahan bergerak dari kesedihan
    menuju harapan yang tenang.",
19                  imageRes = R.drawable.albumcarrieandlowell,
20                  spotifyUrl =
    "https://open.spotify.com/track/1iSI5NeLqGoTsvTk7PA5pY"
21              ),
22              Song(
23                  id = 2,
24                  title = "How to Disappear",
25                  artist = "Lana Del Rey",
26                  album = "Norman Fucking Rockwell!",
27                  year = 2019,
28                  genre = "Dream Pop",
29                  descriptionEn = "A reflective track about
    emotionally unavailable past lovers and learning to let go,
    ending with a sense of peace and self-assurance.",
30                  descriptionId = "Lagu reflektif tentang
    mantan kekasih yang tidak hadir secara emosional, dan
    proses belajar melepaskan, diakhiri dengan rasa damai pada
    diri sendiri.",
31                  imageRes =
    R.drawable.albumnormanfuckingrockwell,
32                  spotifyUrl =
    "https://open.spotify.com/track/4qUtC2BwFC154Ha8YQRrkk"
33              ),
```

```
            Song(
                id = 3,
                title = "Like Real People Do",
                artist = "Hozier",
                album = "From Eden EP",
                year = 2014,
                genre = "Indie Folk",
                descriptionEn = "Inspired by Irish poet Seamus Heaney, the song is about two imperfect people choosing simple, honest intimacy over their complicated pasts.",
                descriptionId = "Terinspirasi dari puisi Seamus Heaney, lagu ini tentang dua orang yang tidak sempurna memilih keintiman sederhana dan jujur daripada masa lalu mereka yang rumit.",
                imageRes = R.drawable.albumfromeden,
                spotifyUrl = "https://open.spotify.com/track/4LGJ2pLDvTRnul3EcZoYkX"
            ),
            Song(
                id = 4,
                title = "The Moon Will Sing",
                artist = "The Crane Wives",
                album = "Coyote Stories",
                year = 2015,
                genre = "Indie Folk",
                descriptionEn = "A bittersweet song about losing one's identity in a relationship, using the moon and sun as a metaphor for emotional dependency.",
                descriptionId = "Lagu bittersweet tentang kehilangan jati diri dalam sebuah hubungan, menggunakan bulan dan matahari sebagai metafora untuk ketergantungan emosional.",
                imageRes = R.drawable.albumherons,
                spotifyUrl = "https://open.spotify.com/track/4v0Zqmuhlbt5nCj712Y26n"
            ),
            Song(
                id = 5,
                title = "Duvet",
                artist = "Bôa",
                album = "Twilight",
                year = 2001,
                genre = "Alternative Rock",
                descriptionEn = "Originally about communication and relying on others, the song became iconic
```

```
as the opening theme of the anime Serial Experiments Lain.",
                descriptionId = "Awalnya tentang komunikasi dan ketergantungan pada orang lain, lagu ini menjadi ikonik sebagai lagu pembuka anime Serial Experiments Lain.",
                imageRes = R.drawable.albumtwilight,
                spotifyUrl = "https://open.spotify.com/track/42qNWdLKCI41S4uzfamhFM"
            ),
            Song(
                id = 6,
                title = "Please Be Rude",
                artist = "Gigi Perez",
                album = "Please Be Rude",
                year = 2024,
                genre = "Indie Pop",
                descriptionEn = "A soft and vulnerable song about the longing for genuine intimacy and the desire to be seen honestly in a romantic relationship.",
                descriptionId = "Lagu yang lembut dan penuh kerentanan tentang kerinduan akan keintiman sejati dan keinginan untuk dilihat apa adanya dalam sebuah hubungan romantis.",
                imageRes = R.drawable.albumpleaseberude,
                spotifyUrl = "https://open.spotify.com/track/2ORD5RHyZI9LGibgqcwr3v"
            ),
            Song(
                id = 7,
                title = "Buzzcut Season",
                artist = "Lorde",
                album = "Pure Heroine",
                year = 2013,
                genre = "Electropop",
                descriptionEn = "A deceptively calm song about teenage escapism and blissful ignorance, while quietly acknowledging the darkness of the outside world.",
                descriptionId = "Lagu yang terdengar tenang namun menyimpan makna dalam tentang pelarian remaja dan kebahagiaan yang naif, sambil diam-diam mengakui kegelapan dunia luar.",
                imageRes = R.drawable.albumpureheroine,
                spotifyUrl = "https://open.spotify.com/track/51QEyJI5M7uyd8DOh9tqQY"
            )
        )
    }
```

```
96  }
```

SongViewModel.kt

Tabel 28. Source Code SongViewModel.kt

```
1   package com.example.songpediacompose.viewmodel
2
3   import android.app.Application
4   import androidx.lifecycle.AndroidViewModel
5   import androidx.lifecycle.LiveData
6   import androidx.lifecycle.MutableLiveData
7   import com.example.songpediacompose.Song
8   import com.example.songpediacompose.data.SongData
9
10  class SongViewModel(application: Application) :
    AndroidViewModel(application) {
11
12      private val _songs = MutableLiveData<List<Song>>()
13      val songs: LiveData<List<Song>> = _songs
14
15      private val _selectedSong = MutableLiveData<Song>()
16      val selectedSong: LiveData<Song> = _selectedSong
17
18      init {
19          _songs.value = SongData.getSongs(application)
20      }
21
22      fun selectSong(song: Song) {
23          _selectedSong.value = song
24      }
25  }
```

Color.kt

Tabel 29. Source Code Color.kt

```
1   package com.example.songpediacompose.ui.theme
2
3   import androidx.compose.ui.graphics.Color
4
5   val Brown = Color(0xFF8B6343)
6   val BrownDark = Color(0xFF4E342E)
7   val BrownLight = Color(0xFFA1887F)
8   val Cream = Color(0xFFF5F0EB)
9   val CardColor = Color(0xFFF0EAE2)
10  val White = Color(0xFFFFFFFF)
11  val TextDark = Color(0xFF3E2723)
```

```
12  val TextBrown = Color(0xFF6D4C41)
```

Theme.kt

Tabel 30. Source Code Theme.kt

```
1   package com.example.songpediacompose.ui.theme
2
3   import androidx.compose.material3.MaterialTheme
4   import androidx.compose.material3.lightColorScheme
5   import androidx.compose.runtime.Composable
6
7   private val LightColors = lightColorScheme(
8       primary = Brown,
9       onPrimary = White,
10      secondary = BrownLight,
11      onSecondary = White,
12      background = White,
13      onBackground = TextDark,
14      surface = CardColor,
15      onSurface = TextDark,
16  )
17
18  @Composable
19  fun SongPediaTheme(content: @Composable () -> Unit) {
20      MaterialTheme(
21          colorScheme = LightColors,
22          typography = Typography,
23          content = content
24      )
25  }
```

Type.kt

Tabel 31. Source Code Type.kt

```
1   package com.example.songpediacompose.ui.theme
2
3   import androidx.compose.material3.Typography
4   import androidx.compose.ui.text.TextStyle
5   import androidx.compose.ui.text.font.FontFamily
6   import androidx.compose.ui.text.font.FontWeight
7   import androidx.compose.ui.unit.sp
8
9   // Set of Material typography styles to start with
10  val Typography = Typography(
11      bodyLarge = TextStyle(
12          fontFamily = FontFamily.Default,
```

```
        fontWeight = FontWeight.Normal,
        fontSize = 16.sp,
        lineHeight = 24.sp,
        letterSpacing = 0.5.sp
    )
    /* Other default text styles to override
    titleLarge = TextStyle(
        fontFamily = FontFamily.Default,
        fontWeight = FontWeight.Normal,
        fontSize = 22.sp,
        lineHeight = 28.sp,
        letterSpacing = 0.sp
    ),
    labelSmall = TextStyle(
        fontFamily = FontFamily.Default,
        fontWeight = FontWeight.Medium,
        fontSize = 11.sp,
        lineHeight = 16.sp,
        letterSpacing = 0.5.sp
    )
    */
)
```

strings.xml

Tabel 32. Source Code MainActivity.kt

```
<resources>
    <string name="app_name">SongPedia</string>
    <string name="home_title">SongPedia</string>
    <string name="label_album">Album</string>
    <string name="label_genre">Genre</string>
    <string name="label_description">Description</string>
    <string name="label_choose_language">Choose Language</string>
    <string name="label_indonesia">Bahasa Indonesia</string>
    <string name="label_english">English</string>
    <string name="btn_spotify">Spotify</string>
    <string name="btn_detail">Detail</string>
    <string name="label_highlighted">Highlighted Songs</string>
    <string name="label_all_songs">All Songs</string>
    <!-- legacy keys -->
    <string name="highlighted_songs">Highlighted Songs</string>
    <string name="all_songs">All Songs</string>
    <string name="detail">Detail</string>
```

```
    <string name="choose_language">Choose Language</string>
</resources>
```

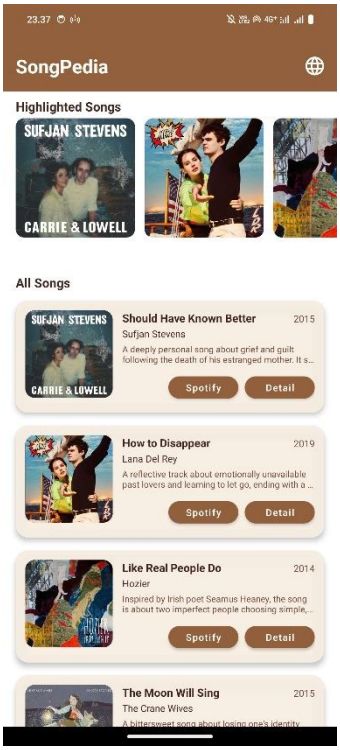
strings.xml (in)

Tabel 33. Source Code strings.kt (in)

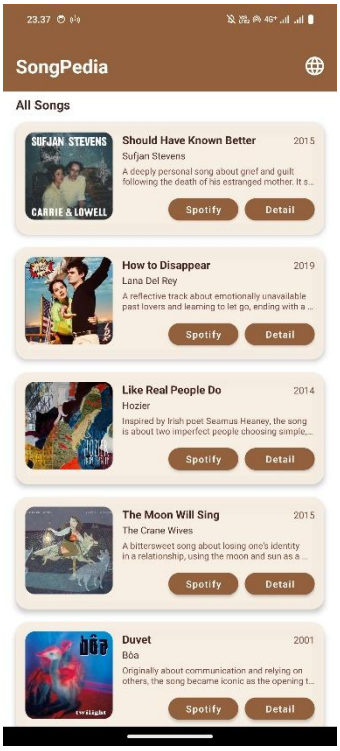
```
<resources>
    <string name="app_name">SongPedia</string>
    <string name="home_title">SongPedia</string>
    <string name="label_album">Album</string>
    <string name="label_genre">Genre</string>
    <string name="label_description">Deskripsi</string>
    <string name="label_choose_language">Pilih Bahasa</string>
    <string name="label_indonesia">Bahasa Indonesia</string>
    <string name="label_english">English</string>
    <string name="btn_spotify">Spotify</string>
    <string name="btn_detail">Detail</string>
    <string name="label_highlighted">Lagu Pilihan</string>
    <string name="label_all_songs">Semua Lagu</string>
    <!-- legacy keys agar tidak error jika masih dipakai -->
    <string name="highlighted_songs">Lagu Pilihan</string>
    <string name="all_songs">Semua Lagu</string>
    <string name="detail">Detail</string>
    <string name="choose_language">Pilih Bahasa</string>
</resources>
```

B. Output Program

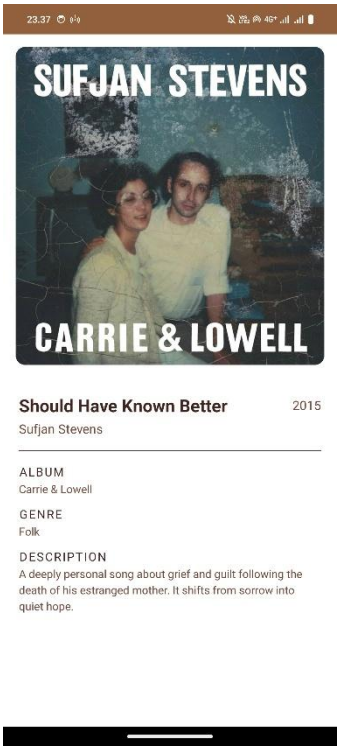
SongPediaXML



Gambar 4. Screenshot Hasil Jawaban Soal 1 XML

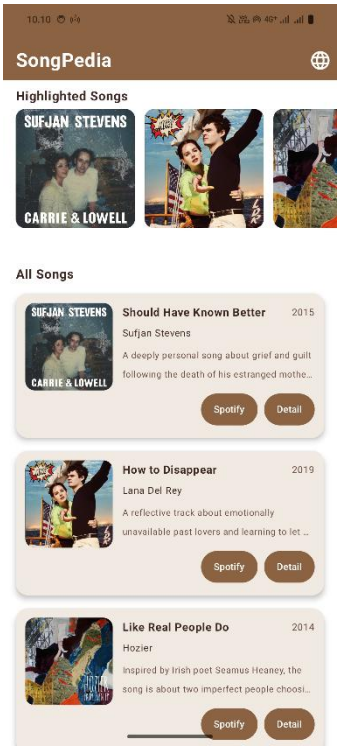


Gambar 5. Screenshot Hasil Jawaban Soal 1 XML

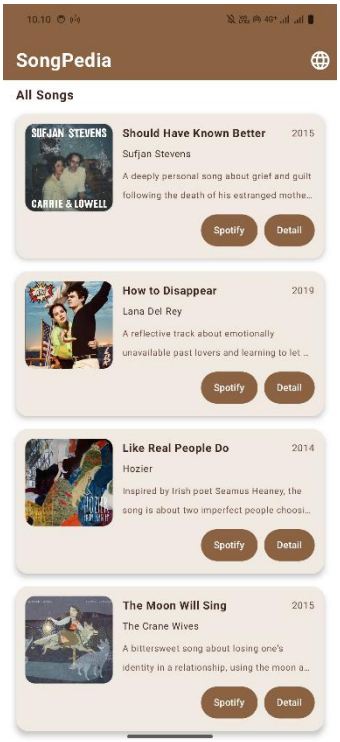


Gambar 6. Screenshot Hasil Jawaban Soal 1 XML

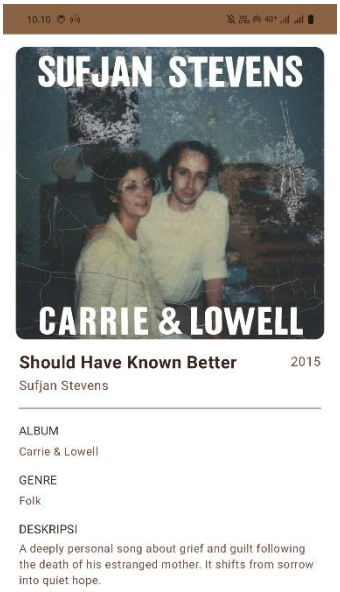
Gambar 7. Screenshot Hasil Jawaban Soal 1 XML



Gambar 8. Screenshot Hasil Jawaban Soal 1 XML



Gambar 9. Screenshot Hasil Jawaban Soal 1 XML



Gambar 10. Screenshot Hasil Jawaban Soal 1 XML

Gambar 11. Screenshot Hasil Jawaban Soal 1 XML

C. Pembahasan

XML

MainActivity.kt (XML)

File 'MainActivity.kt' merupakan satu-satunya Activity dalam aplikasi 'SongPedia' berbasis XML. Class MainActivity mewarisi 'AppCompatActivity' dan menggunakan 'ViewBinding' melalui properti 'binding' bertipe 'ActivityMainBinding'. Di dalam 'onCreate()', hal pertama yang dilakukan adalah menonaktifkan dark mode menggunakan 'AppCompatDelegate.setDefaultNightMode(MODE_NIGHT_NO)'. Selanjutnya, aplikasi membaca 'SharedPreferences' dengan key '"settings"' untuk mengecek apakah bahasa sudah pernah disimpan — jika belum, bahasa default '"id"' langsung disimpan. Bahasa yang tersimpan kemudian diterapkan ke 'Locale' dan 'Configuration' agar seluruh string resource tampil sesuai bahasa yang dipilih. Terakhir, 'binding' diinisialisasi dan setContentView dipanggil menggunakan 'binding.root'.

HomeFragment.kt

File 'HomeFragment.kt' menangani tampilan utama aplikasi yang menampilkan daftar lagu. Fragment ini menggunakan 'ViewBinding' melalui 'FragmentHomeBinding' dengan pola '_binding' nullable untuk menghindari memory leak di 'onDestroyView()'. ViewModel dibagikan antar fragment menggunakan 'activityViewModels()' sehingga data tetap konsisten. Pada 'onViewCreated()', dua 'RecyclerView' disiapkan — 'rvSongs' dengan 'LinearLayoutManager' vertikal untuk daftar lagu, dan 'rvCarousel' dengan 'LinearLayoutManager' horizontal untuk carousel. Keduanya menggunakan adapter masing-masing yaitu 'SongAdapter' dan 'CarouselAdapter'. Data diobservasi dari 'viewModel.songs' dan setiap perubahan langsung memperbarui kedua adapter. Tombol 'btnLanguage' dikonfigurasi untuk menavigasi ke 'LanguageFragment' menggunakan 'Navigation Component'.

DetailFragment.kt

File 'DetailFragment.kt' menampilkan informasi lengkap dari lagu yang dipilih. Fragment ini mengobservasi 'selectedSong' dari ViewModel yang diisi saat pengguna menekan tombol Detail di HomeFragment. Bahasa aktif dibaca dari 'SharedPreferences' dan digunakan untuk menentukan apakah 'descriptionId' atau 'descriptionEn' yang ditampilkan. Data lagu seperti judul, artis, tahun, 'album', dan 'genre' langsung di-set ke masing-masing 'TextView'. Gambar 'album' dimuat menggunakan library 'Glide' dengan transformasi 'RoundedCorners(32)' untuk menghasilkan sudut melengkung pada gambar.

LanguageFragment.kt

File 'LanguageFragment.kt' menangani halaman pemilihan bahasa. Saat fragment dibuat, bahasa yang sedang aktif dibaca dari 'SharedPreferences' lalu 'updateCheckmarks()' dipanggil untuk menampilkan ikon centang pada pilihan yang sesuai. Dua 'MaterialCardView' — 'cardIndonesia' dan 'cardEnglish' — masing-masing memiliki 'setOnClickListener' yang menyimpan pilihan bahasa ke 'SharedPreferences' lalu memanggil fungsi 'setLocale()'. Fungsi ini menerapkan 'Locale' baru ke 'Configuration' dan memanggil 'requireActivity()'.'recreate()' agar seluruh tampilan dirender ulang dengan bahasa baru.

SongAdapter.kt

File 'SongAdapter.kt' adalah adapter 'RecyclerView' untuk menampilkan daftar lagu. Adapter menerima tiga parameter: list songs, lambda 'onDetailClick', dan lambda 'onSpotifyClick'. Di dalam 'onBindViewHolder()', data lagu di-set ke masing-masing view, dan bahasa aktif dibaca dari 'SharedPreferences' untuk memilih antara 'descriptionId' atau 'descriptionEn'. Gambar dimuat menggunakan 'Glide' dengan 'RoundedCorners(24)'. Kedua tombol 'btnSpotify' dan 'btnDetail' dikonfigurasi dengan listener yang meneruskan objek Song ke lambda yang dikirim dari HomeFragment.

CarouselAdapter.kt

File 'CarouselAdapter.kt' adalah adapter sederhana untuk 'RecyclerView' horizontal yang berfungsi sebagai carousel di bagian atas HomeFragment. Adapter hanya menerima list songs dan di dalam 'onBindViewHolder()' memuat gambar 'album' menggunakan 'Glide' dengan 'RoundedCorners(32)' ke 'imgCarousel'. T'id'ak ada listener atau logika tambahan karena carousel berfungsi murni sebagai elemen visual.

SongViewModel.kt

File 'SongViewModel.kt' menggunakan arsitektur 'AndroidViewModel' sehingga memiliki akses ke Application context yang dibutuhkan oleh SongData. Terdapat dua 'MutableLiveData' — '_songs' untuk menyimpan seluruh daftar lagu, dan '_selectedSong' untuk menyimpan lagu yang sedang dipilih. Keduanya diekspos sebagai 'LiveData' yang hanya bisa dibaca dari luar. Saat ViewModel pertama dibuat, blok 'init' langsung memanggil SongData.'getSongs()' untuk mengisi data. Fungsi 'selectSong()' digunakan oleh HomeFragment untuk menetapkan lagu yang akan ditampilkan di DetailFragment.

Song.kt

File 'Song.kt' mendefinisikan model data utama aplikasi sebagai data class. Terdapat sepuluh properti: 'id', 'title', 'artist', 'album', 'year', 'genre', 'descriptionEn', 'descriptionId', 'imageRes', dan 'spotifyUrl'. Adanya dua properti deskripsi ('descriptionEn' dan 'descriptionId') memungkinkan aplikasi menampilkan teks sesuai bahasa yang dipilih tanpa perlu memanggil API atau memuat file tambahan.

SongData.kt

File 'SongData.kt' adalah object singleton yang berfungsi sebagai sumber data statis aplikasi. Fungsi 'getSongs()' menerima Context dan mengembalikan list berisi tujuh objek Song yang sudah diisi lengkap. Setiap item menyertakan referensi gambar dari drawable ('R.drawable'.*) dan URL Spotify yang val'id' untuk membuka lagu di browser atau aplikasi Spotify.

activity_main.xml

File 'activity_main.xml' mendefinisikan tampilan root Activity yang hanya berisi satu 'FragmentContainerView' dengan ID 'navHostFragment'. Komponen ini bertindak sebagai host navigasi yang dikonfigurasi dengan 'app:navGraph'="@navigation/nav_graph" dan 'app:defaultNavHost'="'true'", sehingga seluruh perpindahan antar fragment dikendalikan oleh 'Navigation Component'. Latar belakang Activity diset ke '#12121E' namun t'id'ak terlihat karena seluruh area tertutup oleh fragment.

fragment_home.xml

File 'fragment_home.xml' menggunakan 'LinearLayout' vertikal sebagai root. Di bagian atas terdapat header dengan background="'#8B6343'" yang memuat 'TextView' judul dan 'MaterialButton' dengan ikon language untuk navigasi ke halaman bahasa. Di bawah header, 'NestedScrollView' membungkus konten yang dapat digulir, berisi label "Lagu Pilihan", 'RecyclerView' horizontal ('rvCarousel') dengan tinggi '180dp' untuk carousel, label "Semua Lagu", dan 'RecyclerView' vertikal ('rvSongs') untuk daftar lagu.

fragment_detail.xml

File 'fragment_detail.xml' menggunakan 'ScrollView' agar konten detail dapat digulir. Gambar 'album' ditampilkan menggunakan 'ImageView' dengan tinggi '400dp', margin '16dp' di semua sisi, dan 'clipToOutline="true'" yang bekerja bersama 'rounded_image_bg' drawable untuk menghasilkan sudut melengkung. Di bawah gambar, 'LinearLayout' dengan padding '20dp' menampung judul, tahun, artis, garis pemisah, serta label dan nilai Album, Genre, dan Deskripsi.

fragment_language.xml

File 'fragment_language.xml' menggunakan 'LinearLayout' dengan 'gravity="center'" dan 'padding="32dp'" sehingga seluruh konten terpusat di layar. Di dalamnya terdapat 'TextView' judul, dan dua 'MaterialCardView' dengan 'cardCornerRadius'="'16dp'" dan 'cardBackgroundColor'="'#EDE0D4'" untuk pilihan Bahasa Indonesia dan English. Masing-

masing card memuat 'TextView' nama bahasa dan 'ImageView' ikon centang yang visibilitasnya dikendalikan secara programatik dari 'LanguageFragment'.

item_song.xml

File 'item_song.xml' mendefinisikan tampilan satu item dalam daftar lagu menggunakan 'MaterialCardView' dengan 'cardCornerRadius'="'16dp"' dan 'cardElevation'="'4dp"'. Di dalamnya terdapat 'LinearLayout' horizontal yang memuat 'ImageView' 'album' berukuran 110x'110dp' dengan sudut melengkung, diikuti 'LinearLayout' vertikal berisi judul dan tahun di baris pertama, artis di baris kedua, deskripsi dengan 'maxLines'="'2"', serta dua 'MaterialButton' — 'btnSpotify' dan 'btnDetail' — yang rata kanan dengan 'cornerRadius'="'20dp"' dan warna '#8B6343'.

item_carousel.xml

File 'item_carousel.xml' mendefinisikan satu item carousel berukuran 150x'150dp'. Di dalamnya hanya terdapat satu 'ImageView' dengan ID 'imgCarousel' yang mengisi seluruh area dengan 'scaleType="centerCrop"' dan 'clipToOutline="true"' untuk menerapkan sudut melengkung dari 'rounded_image_bg'.

nav_graph.xml

File 'nav_graph.xml' mendefinisikan seluruh alur navigasi aplikasi. Start destination diset ke 'homeFragment'. Dari 'homeFragment' terdapat dua action: 'action_home_to_detail' yang mengarah ke 'detailFragment', dan 'action_home_to_language' yang mengarah ke 'languageFragment'. Kedua fragment tujuan t'id'ak memiliki action keluar karena navigasi kembali ditangani oleh back stack secara otomatis.

strings.xml

File 'strings.xml' menyimpan seluruh konten teks aplikasi sebagai resource string. Terdapat dua versi file ini — satu di 'res/values/' untuk bahasa Indonesia, dan satu di 'res/values-en/' untuk bahasa Inggris. Perbedaan utama keduanya ada pada 'label_description' ("Deskripsi" vs "Description"), 'label_choose_language', 'label_highlighted', dan 'label_all_songs'. Andro'id' secara otomatis memilih file yang sesuai berdasarkan 'Locale' yang sedang aktif.

themes.xml dan colors.xml

File 'themes.xml' mendefinisikan tema aplikasi menggunakan 'Theme.MaterialComponents.Light.NoActionBar' sebagai parent, dengan 'colorPrimary' diset ke coklat tua '#3E2723' dan 'android:statusBarColor' ke '#8B6343' agar warna status bar selaras dengan header. File 'colors.xml' hanya mendefinisikan dua warna dasar black dan white karena sebagian besar warna dalam aplikasi ini d'id'efinisikan langsung secara inline di masing-masing layout XML.

COMPOSE

MainActivity.kt

File 'MainActivity.kt' pada versi Compose mewarisi 'ComponentActivity', berbeda dengan versi XML yang menggunakan 'AppCompatActivity'. Di dalam 'onCreate()', langkah pertama adalah menonaktifkan dark mode menggunakan 'AppCompatDelegate.setDefaultNightMode(MODE_NIGHT_NO)'. Selanjutnya, SharedPreferences dengan key '"settings"' dibaca untuk memeriksa apakah bahasa sudah tersimpan — jika belum, nilai default '"id"' langsung disimpan. Bahasa yang tersimpan kemudian diterapkan ke 'Locale' dan 'Configuration' agar string resource tampil sesuai bahasa yang dipilih. Setelah itu, 'enableEdgeToEdge()' dipanggil agar tampilan meluas hingga area status bar, lalu 'setContent' digunakan untuk mendefinisikan UI secara deklaratif. Di dalamnya, 'SongPediaTheme' membungkus 'Surface' berwarna coklat '#8B6343' sebagai latar belakang, dan 'Box' dengan modifier 'statusBarsPadding()' memastikan konten tidak tertutup status bar. Di dalam 'Box', 'NavGraph()' dipanggil sebagai entry point seluruh navigasi aplikasi.

NavGraph.kt

File 'NavGraph.kt' mendefinisikan seluruh alur navigasi aplikasi Compose menggunakan Navigation Component. Fungsi composable 'NavGraph()' menginisialisasi 'navController' melalui 'rememberNavController()' dan satu instance 'SongViewModel' yang dibagikan ke seluruh screen menggunakan 'viewModel()'. 'NavHost' dikonfigurasi dengan start destination '"home"' dan mendaftarkan tiga route: '"home"' yang merender 'HomeScreen', '"detail"' yang merender 'DetailScreen', dan '"language"' yang merender 'LanguageScreen'. Navigasi antar screen dilakukan

melalui 'navController.navigate()' untuk berpindah maju dan 'navController.popBackStack()' untuk kembali ke screen sebelumnya.

HomeScreen.kt

File 'HomeScreen.kt' memuat dua fungsi composable utama, yaitu 'HomeScreen' dan 'SongItem'. Di level file, tiga warna didefinisikan sebagai properti top-level: 'HeaderColor', 'ButtonBrown', dan 'CardBg'. Fungsi 'HomeScreen' menerima tiga parameter: 'viewModel', 'onDetailClick', dan 'onLanguageClick'. Data lagu diobservasi dari 'viewModel.songs' menggunakan 'observeAsState()', dan bahasa aktif dibaca dari SharedPreferences. Struktur utama layar dibangun menggunakan 'Column' yang terdiri dari dua bagian: header dan 'LazyColumn'. Header berupa 'Row' dengan latar belakang coklat yang memuat 'Text' judul dan 'IconButton' berisi ikon 'Icons.Outlined.Language' untuk navigasi ke halaman bahasa. Di dalam 'LazyColumn', blok 'item' pertama menampilkan label "Lagu Pilihan", 'LazyRow' berisi carousel gambar album berukuran '150dp' dengan 'RoundedCornerShape(12.dp)', lalu label "Semua Lagu". Setelah blok 'item', fungsi 'items(songs)' merender setiap lagu menggunakan composable 'SongItem'.

Fungsi 'SongItem' merender satu item lagu dalam 'Card' dengan 'RoundedCornerShape(16.dp)' dan elevasi '4dp'. Di dalamnya terdapat 'Row' yang memuat gambar album berukuran '110dp' dimuat menggunakan 'AsyncImage' dari library Coil, diikuti 'Column' berisi judul dan tahun di baris pertama, nama artis di baris kedua, deskripsi dengan 'maxLines = 2', serta dua 'Button' yang rata kanan menggunakan 'Arrangement.End' — satu untuk membuka Spotify via intent eksplisit dan satu untuk navigasi ke halaman detail.

DetailScreen.kt

File 'DetailScreen.kt' mendefinisikan composable 'DetailScreen' yang menampilkan informasi lengkap lagu yang dipilih. Data lagu diobservasi dari 'viewModel.selectedSong' dan seluruh konten hanya dirender jika nilainya tidak null. Gambar album dimuat menggunakan 'AsyncImage' dengan tinggi '400dp', padding '16dp' di semua sisi, dan 'clip(RoundedCornerShape(12.dp))' untuk sudut melengkung. Di bawah gambar, 'Column' dengan padding horizontal '20dp' menampilkan judul dan tahun dalam satu 'Row', nama artis, lalu garis pemisah yang digambar menggunakan 'Canvas' dengan 'drawRect' berwarna '#3E2723'. Setelahnya, label Album, Genre, dan Deskripsi

ditampilkan menggunakan '.uppercase()' dan 'letterSpacing = 0.1.sp' tanpa bold, masing-masing diikuti nilai datanya. Pemilihan bahasa deskripsi dilakukan secara kondisional berdasarkan nilai 'lang' yang dibaca dari SharedPreferences.

LanguageScreen.kt

File 'LanguageScreen.kt' mendefinisikan composable 'LanguageScreen' untuk halaman pemilihan bahasa. State bahasa yang sedang aktif disimpan menggunakan 'remember { mutableStateOf(...) }' yang diisi dari SharedPreferences saat pertama kali composable dibuat. Tampilan utama adalah 'Column' dengan 'Arrangement.Center' dan padding '32dp' yang menampilkan judul dan dua 'Card' clickable — satu untuk Bahasa Indonesia dan satu untuk English. Setiap card memuat 'Row' berisi 'Text' nama bahasa dan 'Icon' centang dari 'Icons.Filled.CheckBox' yang hanya ditampilkan jika bahasa tersebut sedang aktif. Saat salah satu card diklik, nilai 'selectedLang' diperbarui, pilihan disimpan ke SharedPreferences, lalu fungsi 'setLocale()' dipanggil. Fungsi 'setLocale()' menerapkan 'Locale' baru ke 'Configuration' dan memanggil 'recreate()' pada Activity agar tampilan dirender ulang dengan bahasa yang baru.

Song.kt

File 'Song.kt' mendefinisikan model data utama aplikasi sebagai 'data class' dengan sepuluh properti: 'id', 'title', 'artist', 'album', 'year', 'genre', 'descriptionEn', 'descriptionId', 'imageRes', dan 'spotifyUrl'. Adanya dua properti deskripsi memungkinkan aplikasi menampilkan teks sesuai bahasa yang dipilih tanpa memerlukan sumber data tambahan.

SongData.kt

File 'SongData.kt' adalah 'object' singleton yang menyimpan seluruh data lagu secara statis. Fungsi 'getSongs()' menerima 'Context' dan mengembalikan list berisi tujuh objek 'Song' yang sudah diisi lengkap, masing-masing dengan referensi gambar dari drawable menggunakan 'R.drawable.*' dan URL Spotify yang valid untuk membuka lagu di browser atau aplikasi Spotify.

SongViewModel.kt

File 'SongViewModel.kt' menggunakan 'AndroidViewModel' sehingga memiliki akses ke 'Application' context yang dibutuhkan oleh 'SongData'. Terdapat dua 'MutableLiveData' —

'_songs' untuk seluruh daftar lagu dan '_selectedSong' untuk lagu yang sedang dipilih — yang masing-masing diekspos sebagai 'LiveData' read-only. Blok 'init' langsung memanggil 'SongData.getSongs()' saat ViewModel pertama dibuat. Fungsi 'selectSong()' digunakan oleh 'HomeScreen' untuk menetapkan lagu yang akan ditampilkan di 'DetailScreen'.

Color.kt

File 'Color.kt' mendefinisikan seluruh warna yang digunakan dalam aplikasi sebagai properti top-level. Terdapat delapan warna: 'Brown' untuk header, 'BrownDark' sebagai cadangan, 'BrownLight' untuk warna sekunder, 'Cream' sebagai cadangan, 'CardColor' untuk latar belakang card, 'White' untuk teks di atas latar gelap, 'TextDark' untuk teks judul dan label, serta 'TextBrown' untuk teks artis dan deskripsi.

Theme.kt

File 'Theme.kt' mendefinisikan tema aplikasi menggunakan 'lightColorScheme' dari Material3. Skema warna dikonfigurasi dalam objek 'LightColors' yang memetakan 'primary' ke 'Brown', 'background' ke 'White', 'surface' ke 'CardColor', dan warna teks masing-masing ke 'TextDark'. Fungsi composable 'SongPediaTheme' membungkus 'MaterialTheme' dengan skema warna dan tipografi yang sudah dikonfigurasi, lalu digunakan di 'MainActivity' sebagai pembungkus seluruh UI.

Type.kt

File 'Type.kt' mendefinisikan konfigurasi tipografi aplikasi menggunakan 'Typography' dari Material3. Hanya satu gaya yang aktif dikonfigurasi, yaitu 'bodyLarge' dengan 'FontFamily.Default', ukuran '16sp', tinggi baris '24sp', dan jarak huruf '0.5.sp'. Gaya lainnya seperti 'titleLarge' dan 'labelSmall' tersedia sebagai komentar contoh yang bisa diaktifkan jika diperlukan.

strings.xml

File 'strings.xml' menyimpan seluruh konten teks aplikasi sebagai resource string. Terdapat dua versi — satu di 'res/values/' untuk bahasa Indonesia dan satu di 'res/values-en/' untuk bahasa Inggris. Perbedaan utama antara keduanya ada pada 'label_description' ("Deskripsi" vs

"Description"), 'label_choose_language', 'label_highlighted', dan 'label_all_songs'. Pada versi Compose juga terdapat beberapa legacy key seperti 'highlighted_songs' dan 'all_songs' yang dipertahankan untuk mencegah error apabila masih ada referensi lama di kode. Android secara otomatis memilih file yang sesuai berdasarkan 'Locale' yang sedang aktif.

SOAL 2

Mengapa RecyclerView masih digunakan padahal RecyclerView memiliki kode yang panjang dan bersifat boiler-plate, dibandingkan LazyColumn dengan kode yang lebih singkat?

A. Pembahasan

RecyclerView masih digunakan meskipun kodenya lebih panjang dan bersifat boilerplate karena memiliki beberapa keunggulan yang tidak dapat diabaikan begitu saja. Alasan pertama adalah kompatibilitas dan relevansi terhadap project yang sudah ada. RecyclerView sudah hadir sejak lama dan menjadi komponen standar dalam pengembangan Android berbasis XML. Banyak project yang masih aktif dikembangkan menggunakan XML dan tidak dapat langsung bermigrasi ke Jetpack Compose, sehingga RecyclerView tetap menjadi pilihan utama yang tidak tergantikan dalam konteks tersebut.

Alasan kedua adalah kematangan ekosistem. RecyclerView didukung oleh dokumentasi yang sangat lengkap, komunitas yang besar, serta banyak library pendukung yang sudah teruji. Fitur-fitur seperti ItemDecoration, ItemAnimator, dan DiffUtil memberikan fleksibilitas tinggi dalam mengelola tampilan list yang kompleks, dan semuanya sudah terbukti stabil digunakan di berbagai skala project nyata.

Alasan ketiga adalah performa yang optimal. RecyclerView hanya me-render item yang sedang terlihat di layar melalui mekanisme view recycling, sehingga penggunaan memori jauh lebih efisien dibandingkan pendekatan list konvensional. Hal ini sangat krusial ketika menampilkan data dalam jumlah besar.

Alasan keempat adalah kontrol yang lebih granular. RecyclerView memberikan kendali penuh kepada developer, mulai dari pengaturan LayoutManager secara custom, animasi per item, hingga gesture seperti swipe dan drag-and-drop yang dapat dikonfigurasi secara detail.

Sementara itu, LazyColumn pada Jetpack Compose memang menawarkan kode yang jauh lebih ringkas dan mudah dibaca, namun hanya dapat digunakan dalam ekosistem Compose. Keduanya tidak dapat saling menggantikan sepenuhnya karena beroperasi pada paradigma yang

berbeda. Dengan demikian, RecyclerView tetap relevan dan penting untuk dikuasai, terutama dalam pengembangan aplikasi Android berbasis XML yang masih mendominasi banyak project di industri saat ini.

.

SOAL 3

Pada desain arsitektur Kotlin, tedapat beberapa arsitektur dengan salah satunya adalah CLEAN ARCHITECTURE, menurut **pendapat kalian**, mengapa kita harus menerapkan arsitektur ini ketika membuat sebuah project?

A. Pembahasan

Clean Architecture perlu diterapkan dalam sebuah project karena memberikan struktur kode yang terorganisir, mudah dipelihara, dan mudah dikembangkan dalam jangka panjang.

Alasan pertama adalah pemisahan tanggung jawab atau separation of concerns. Clean Architecture membagi kode menjadi beberapa lapisan yang memiliki tanggung jawab masing-masing, yaitu lapisan Presentation yang menangani tampilan UI, lapisan Domain yang berisi logika bisnis, dan lapisan Data yang menangani sumber data seperti database atau API. Dengan pemisahan ini, setiap bagian kode memiliki tugas yang jelas dan tidak saling bercampur, sehingga kode menjadi lebih rapi dan mudah dipahami.

Alasan kedua adalah kemudahan dalam pengujian atau testability. Karena setiap lapisan berdiri sendiri dan tidak bergantung langsung satu sama lain, pengujian dapat dilakukan secara independen per lapisan. Logika bisnis di lapisan Domain misalnya, dapat diuji tanpa perlu melibatkan UI atau database, sehingga proses unit testing menjadi jauh lebih mudah dan efisien.

Alasan ketiga adalah kemudahan pemeliharaan dan pengembangan. Ketika terjadi perubahan pada salah satu lapisan, misalnya mengganti sumber data dari database lokal ke API, perubahan tersebut tidak perlu menyentuh lapisan lainnya. Hal ini sangat menguntungkan dalam project besar yang terus berkembang karena risiko munculnya bug akibat perubahan kode menjadi jauh lebih kecil.

Alasan keempat adalah skalabilitas. Clean Architecture memudahkan penambahan fitur baru tanpa mengganggu fitur yang sudah ada. Setiap fitur baru cukup mengikuti struktur lapisan yang sudah ditetapkan, sehingga konsistensi kode tetap terjaga meskipun project terus berkembang dan melibatkan banyak developer.

Dengan demikian, meskipun penerapan Clean Architecture membutuhkan lebih banyak file dan struktur folder yang lebih kompleks di awal, manfaat jangka panjangnya sangat signifikan. Project menjadi lebih mudah dipahami oleh developer baru, lebih mudah diuji, dan lebih siap menghadapi perubahan kebutuhan di masa mendatang.

TAUTAN GIT

Berikut adalah tautan untuk *source code* yang telah dibuat.

https://github.com/arinn-lab/Module-Mobile